

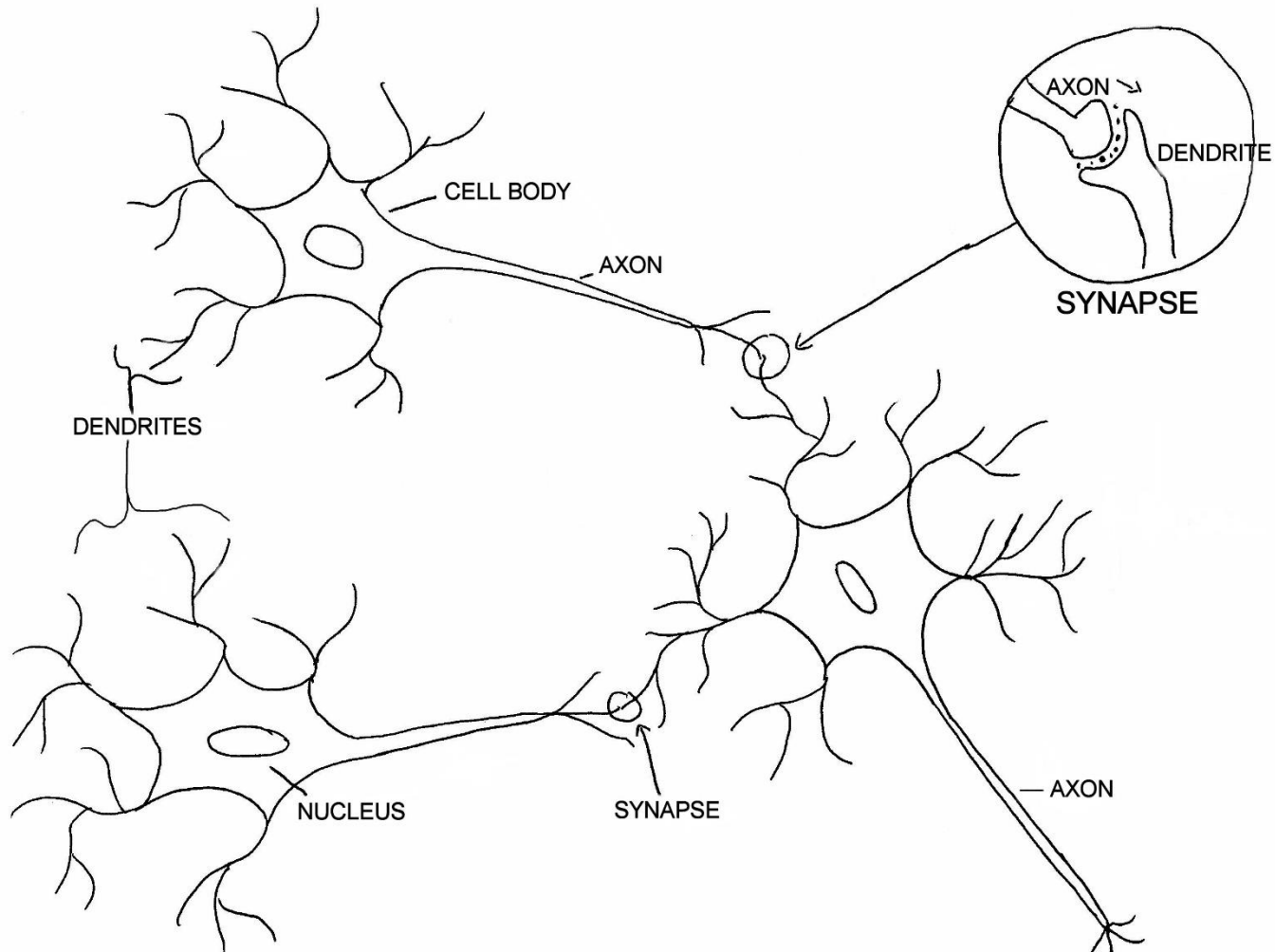
A Brief Overview of MLP

“If the brain were so simple that we could understand it, then we’d be so simple that we couldn’t”

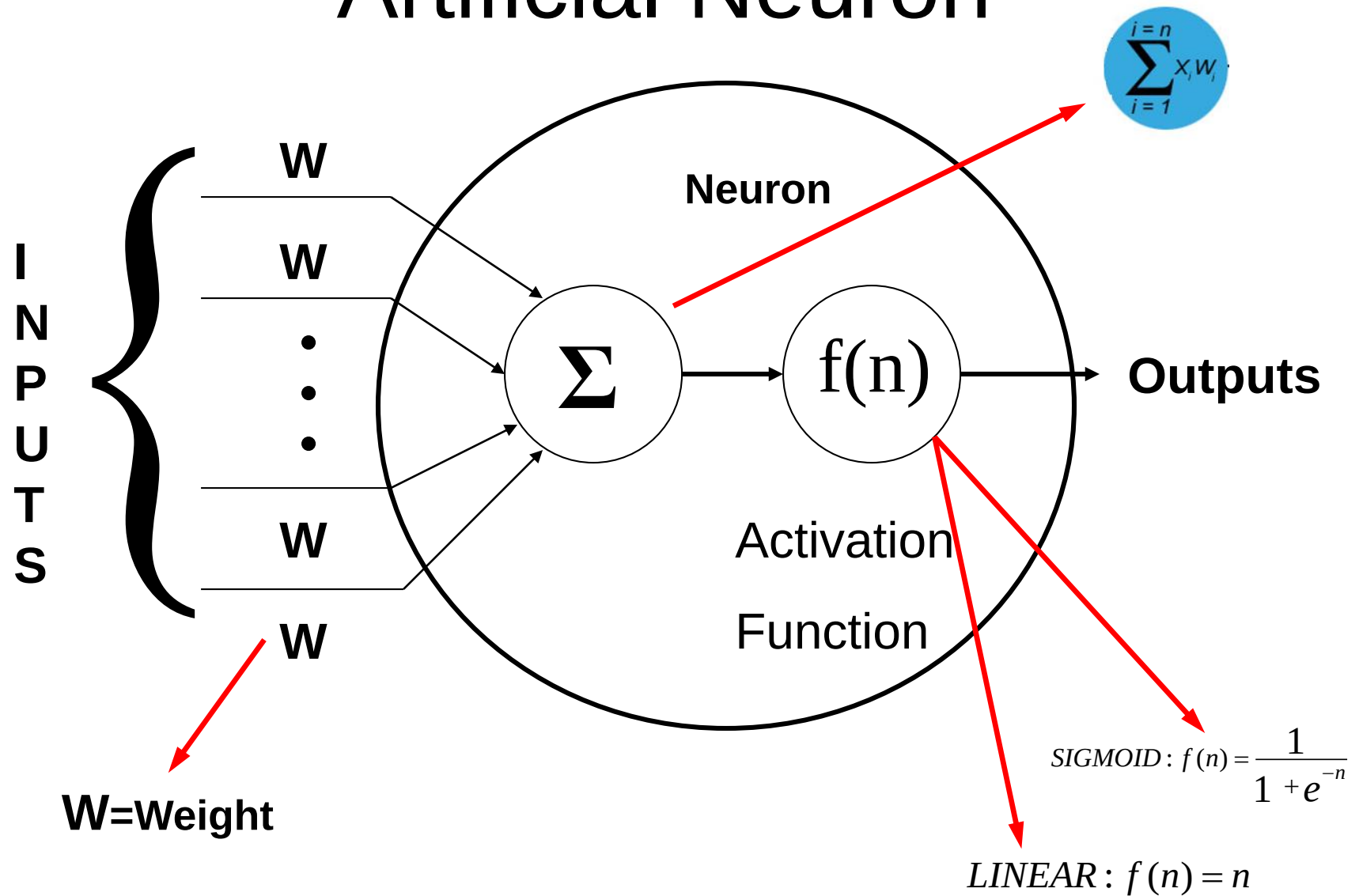
Lyall Watson

Biological Neuron

BIOLOGICAL NEURONS



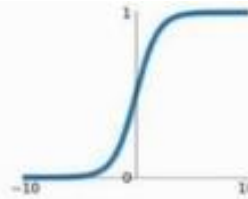
Artificial Neuron



Transfer Functions

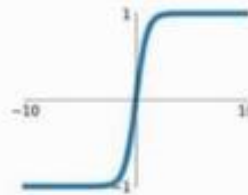
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

$$\tanh(x)$$



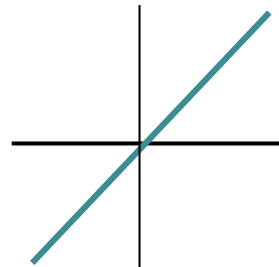
ReLU

$$\max(0, x)$$

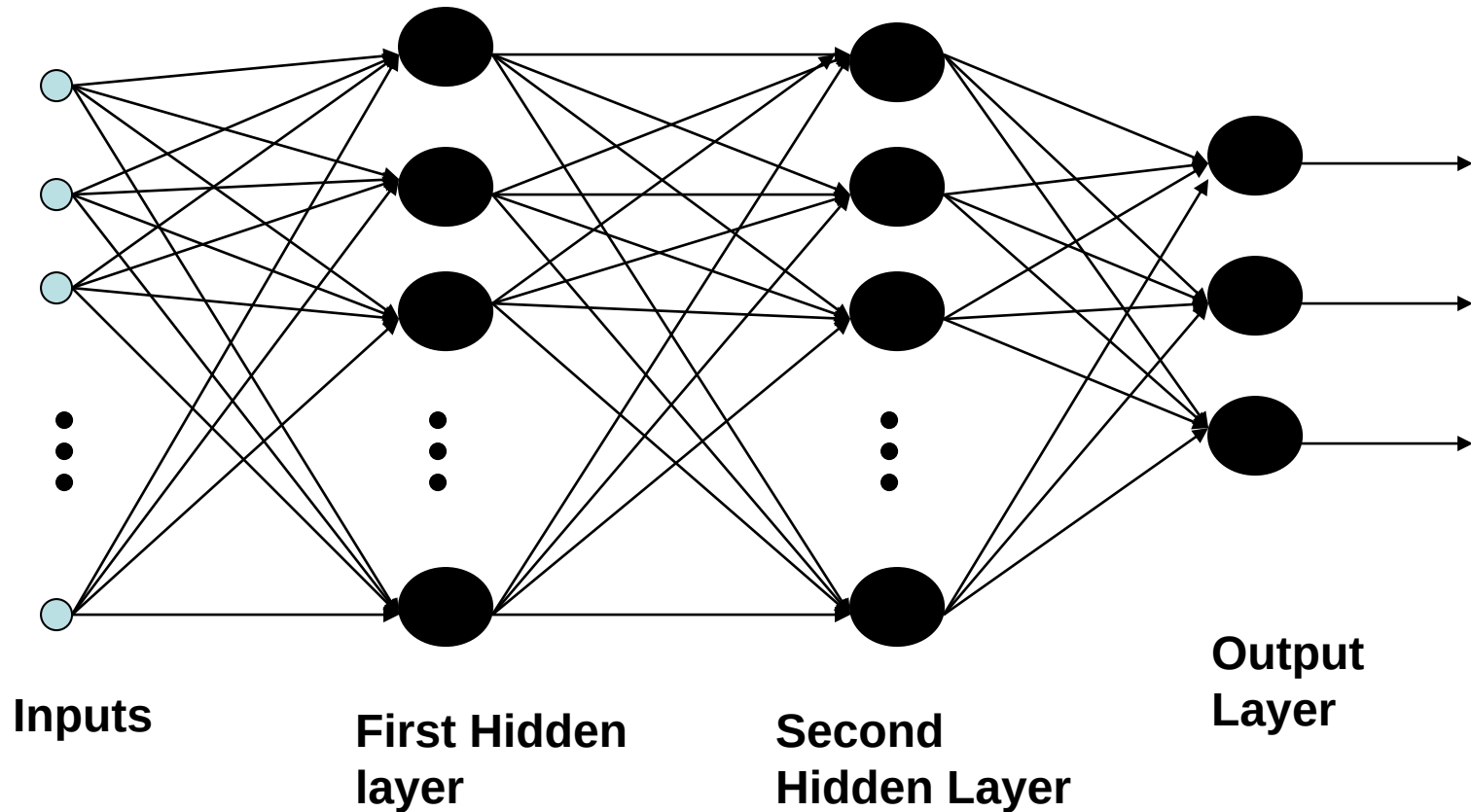


Linear

$$F(x)=x$$



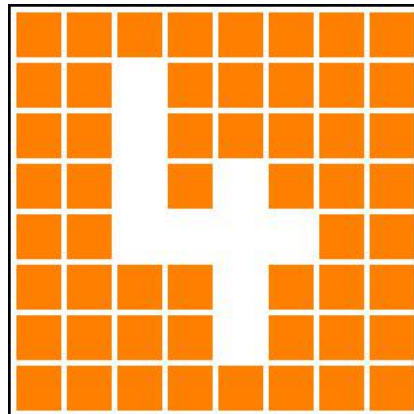
Multilayer Perceptron



Feedforward network: The neurons in each layer feed their output forward to the next layer until we get the final output from the neural network.

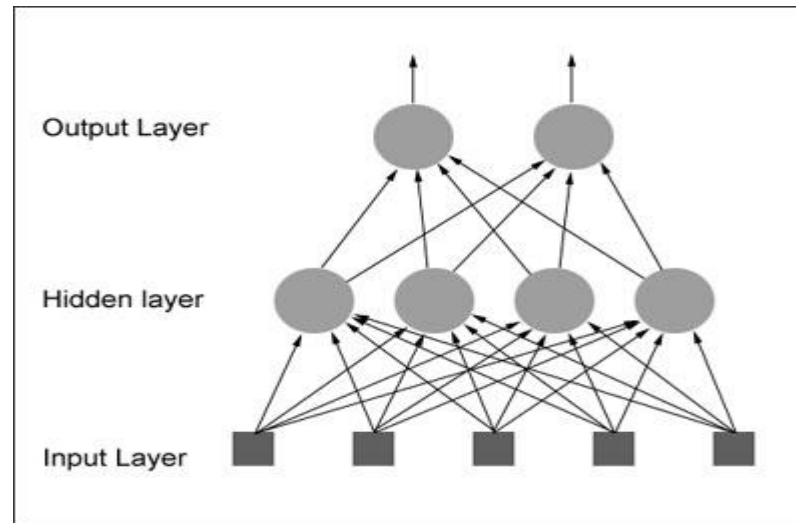
Neural Networks by an Example

- let's design a neural network that will detect the number '4'.
- Given a panel made up of a grid of lights which can be either on or off, we want our neural net to let us know whenever it thinks it sees the character '4'.
- The panel is eight cells square and looks like this:
- the neural net will have **64 inputs**, each one representing a particular cell in the panel and a hidden layer consisting of a number of neurons (more on this later) all feeding their output into just **one neuron in the output** layer



Neural Networks by an Example

- initialize the neural net with random weights
- feed it a series of inputs which represent, in this example, the different panel configurations
- For each configuration we check to see what its output is and adjust the weights accordingly so that whenever it sees something looking like a number 4 it outputs a 1 and for everything else it outputs a zero.



Hidden layers

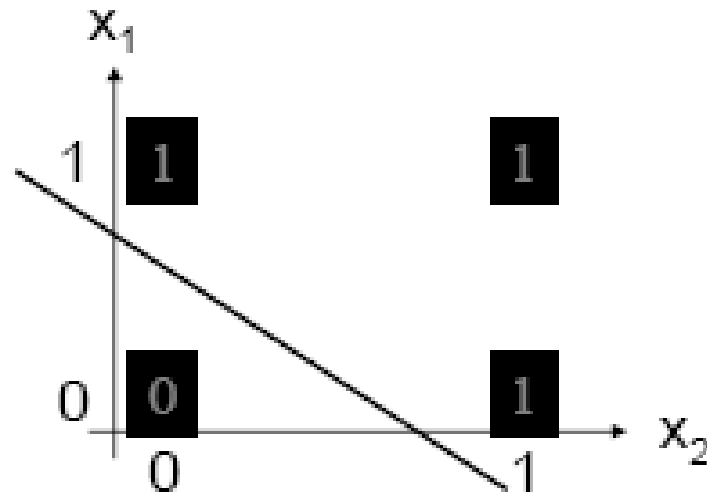
- what do the extra layers gain you? Start with looking at what a single layer can't do

Perceptron Learning Theorem

- A perceptron (threshold unit) can *learn* anything separable with a hyperplane)

OR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

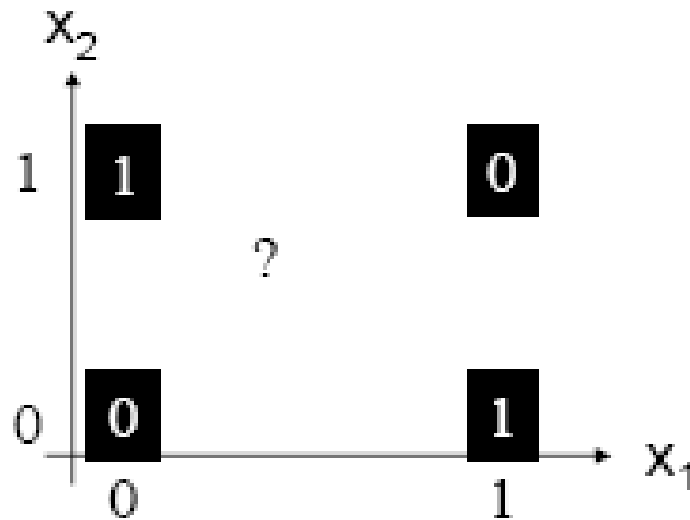


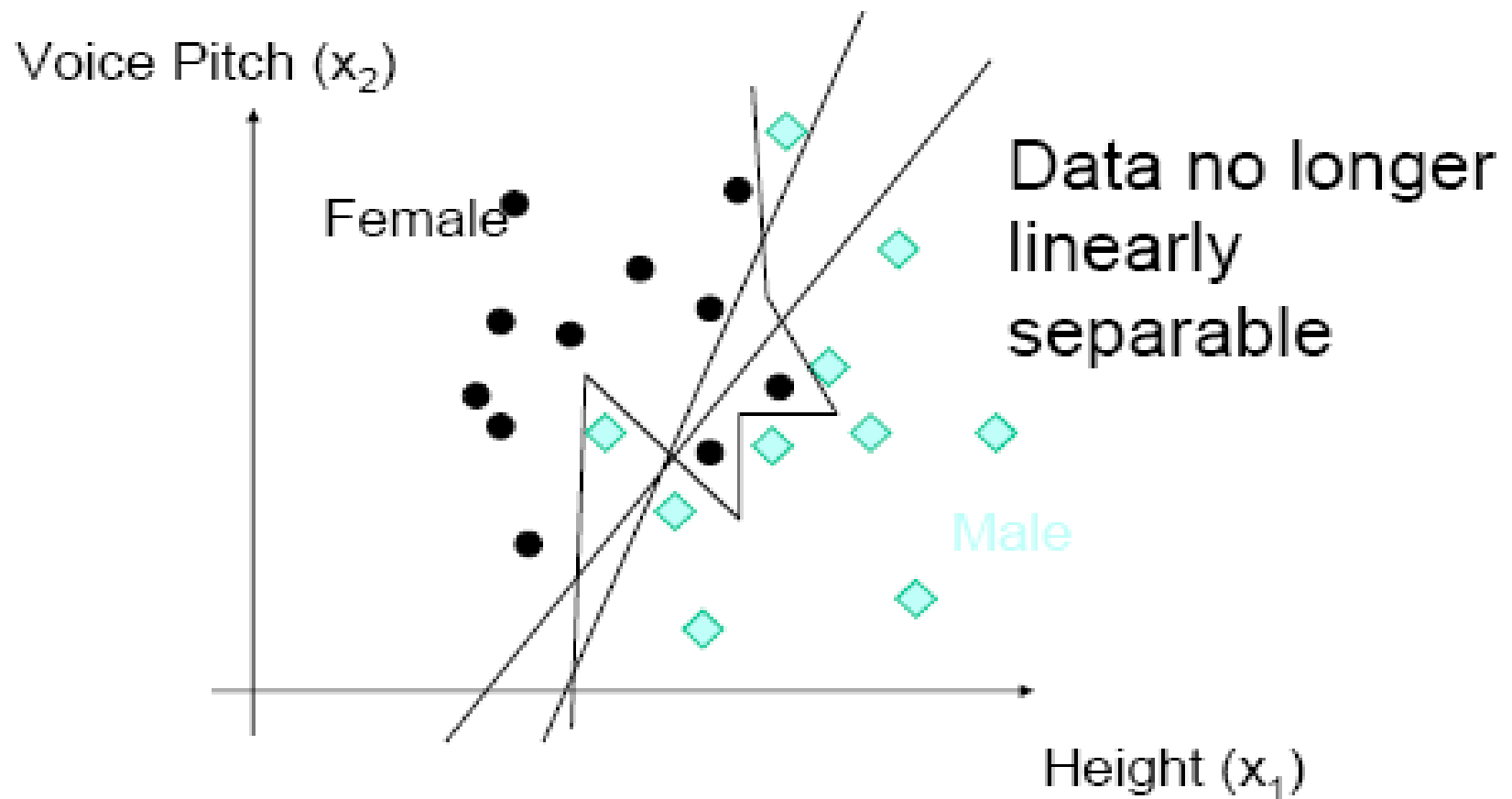
The Exclusive OR problem

A Perceptron cannot represent Exclusive OR since it is not linearly separable.

XOR function

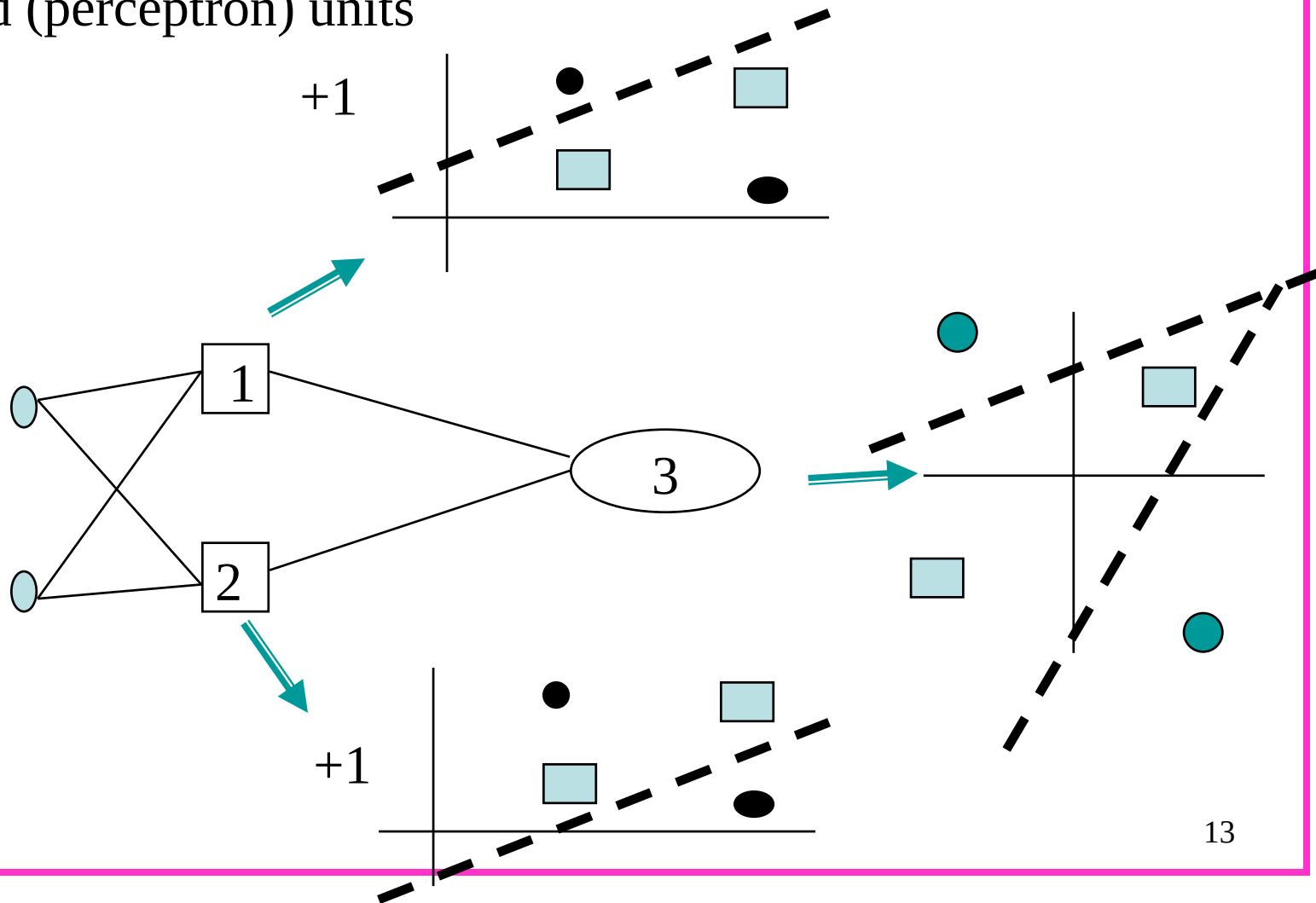
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



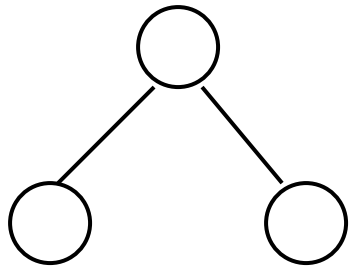
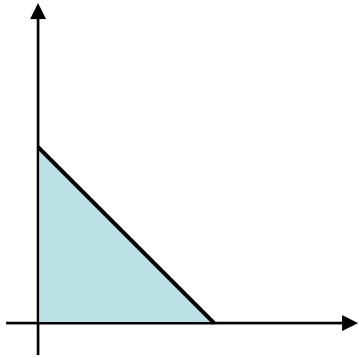


What is a good decision boundary ?

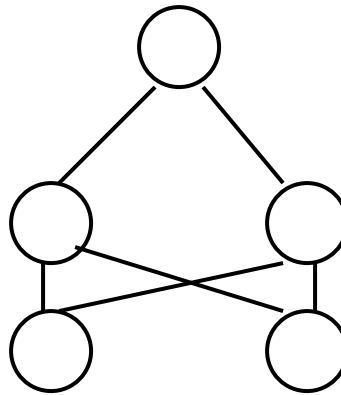
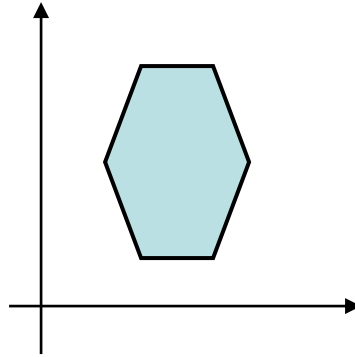
Minsky & Papert (1969) offered solution to XOR problem by combining perceptron unit responses using a second layer of Units. Piecewise linear classification using an MLP with threshold (perceptron) units



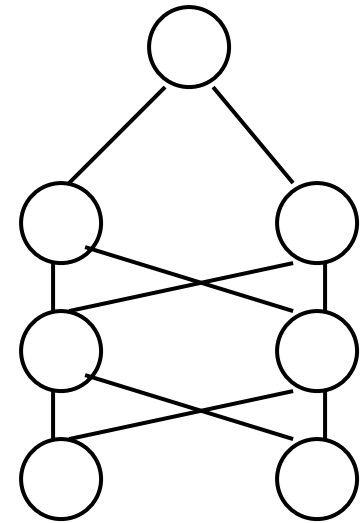
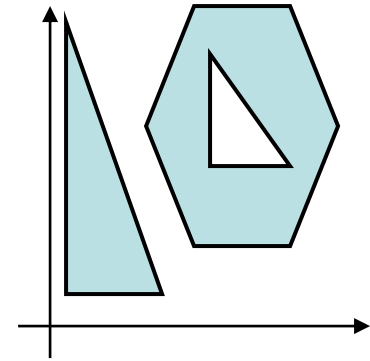
What do each of the layers do?



1st layer draws
linear boundaries



2nd layer combines
the boundaries



3rd layer can generate
arbitrarily complex
boundaries

A dataset

<i>Fields</i>	<i>class</i>
---------------	--------------

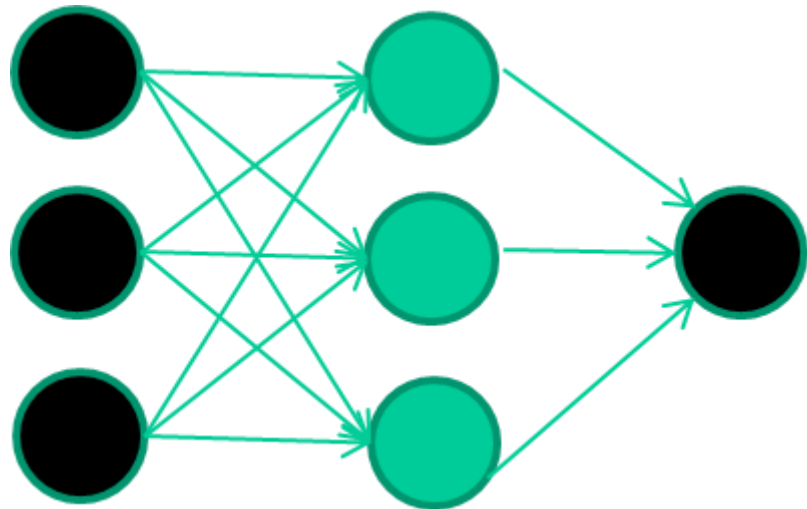
1.4	2.7	1.9	0
-----	-----	-----	---

3.8	3.4	3.2	0
-----	-----	-----	---

6.4	2.8	1.7	1
-----	-----	-----	---

4.1	0.1	0.2	0
-----	-----	-----	---

etc ...



Training the neural network

Fields ***class***

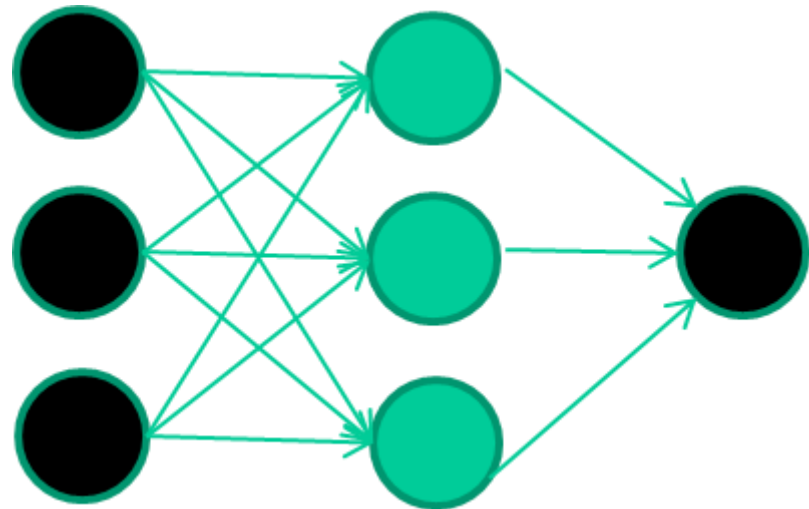
1.4 2.7 1.9 0

3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...



Training data

Fields ***class***

1.4 2.7 1.9 0

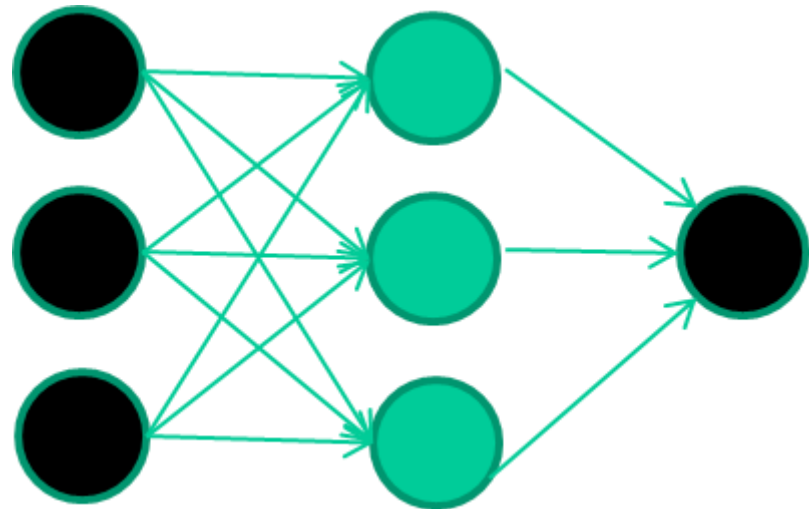
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Initialise with random weights



Training data

Fields ***class***

1.4 2.7 1.9 0

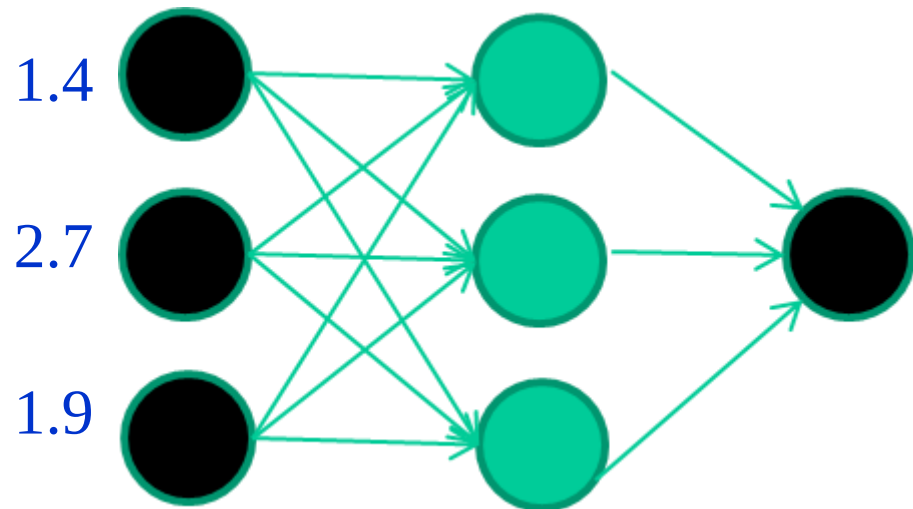
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Present a training pattern



Training data

Fields ***class***

1.4 2.7 1.9 0

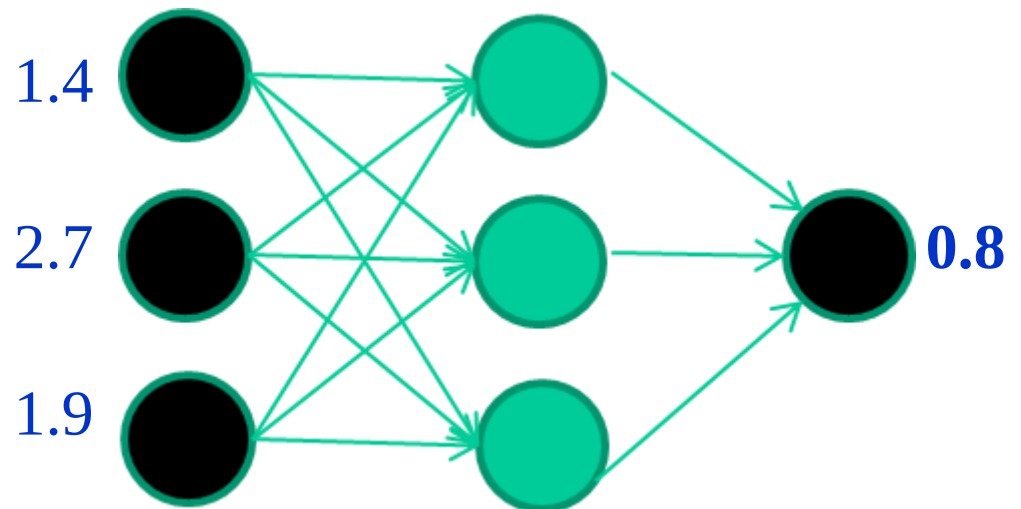
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Feed it through to get output



Training data

Fields ***class***

1.4 2.7 1.9 0

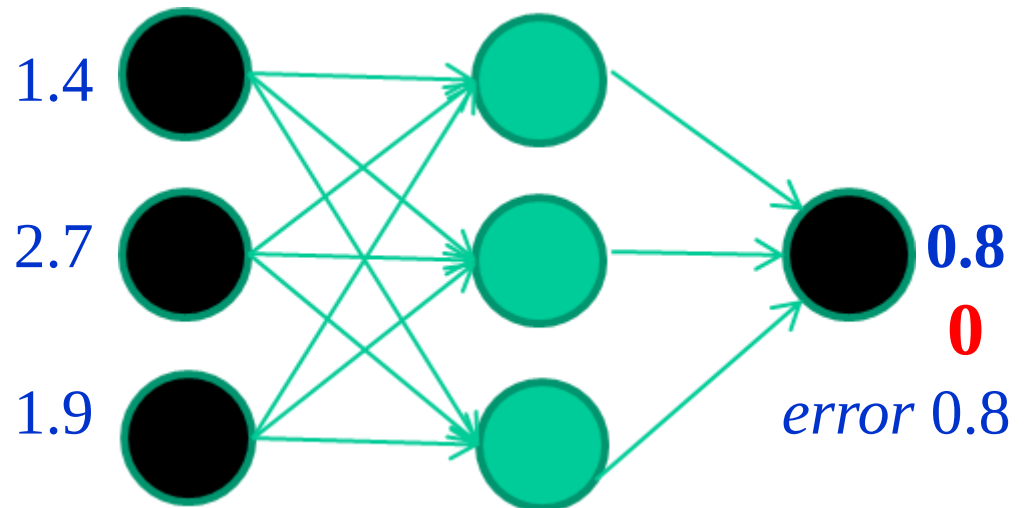
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Compare with target output



Training data

Fields ***class***

1.4 2.7 1.9 0

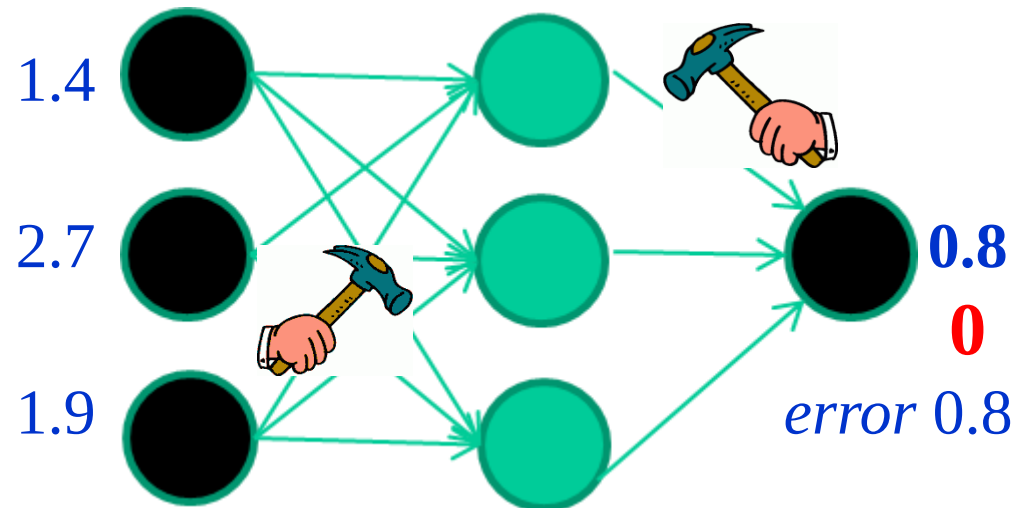
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Adjust weights based on error



Training data

Fields ***class***

1.4 2.7 1.9 0

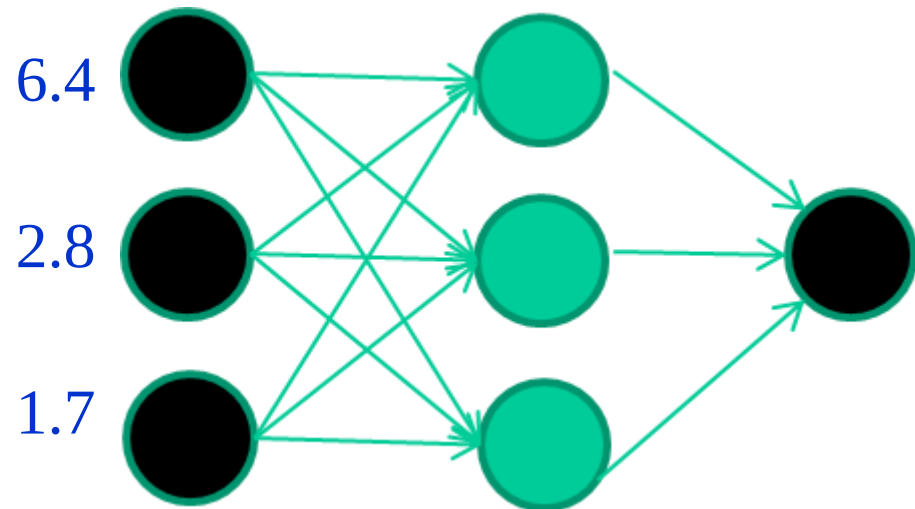
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Present a training pattern



Training data

Fields ***class***

1.4 2.7 1.9 0

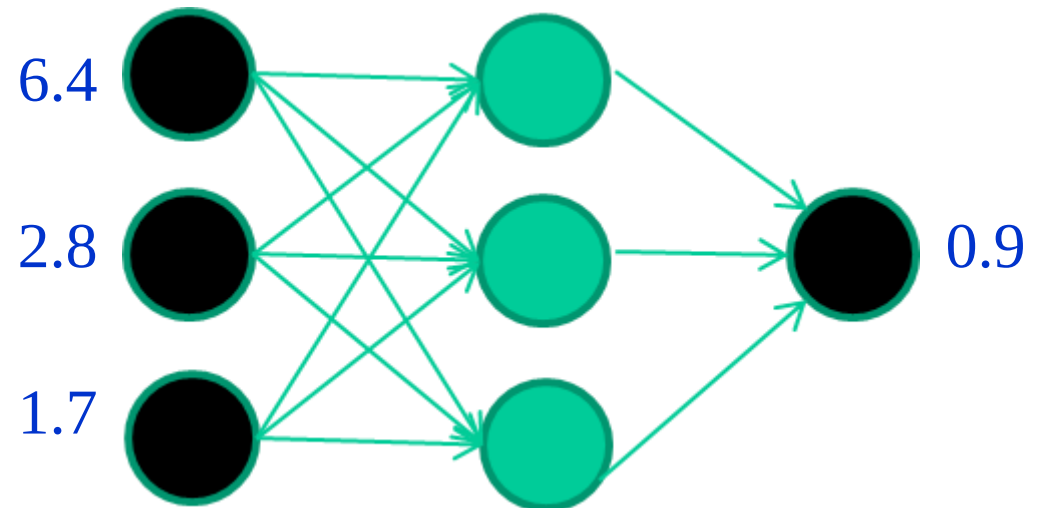
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Feed it through to get output



Training data

Fields ***class***

1.4 2.7 1.9 0

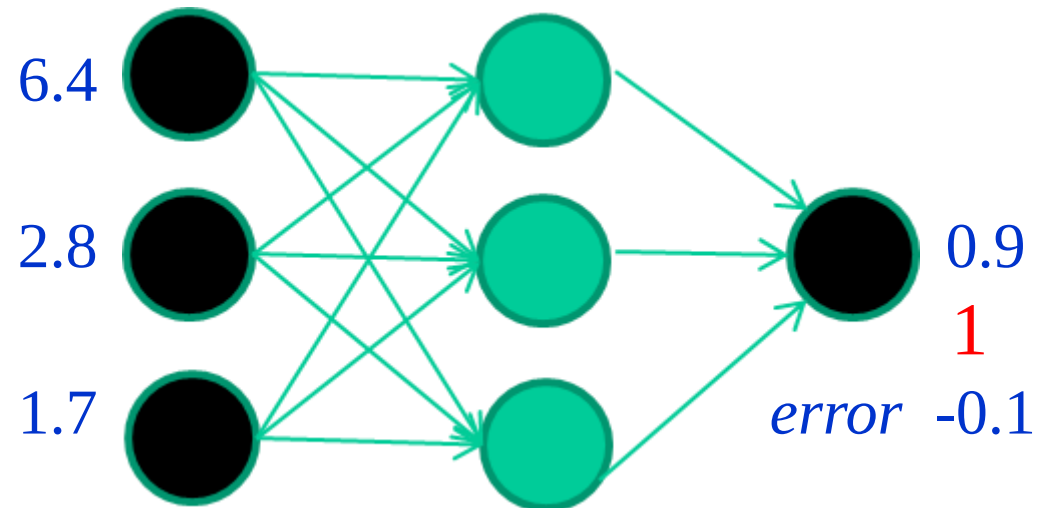
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Compare with target output



Training data

Fields ***class***

1.4 2.7 1.9 0

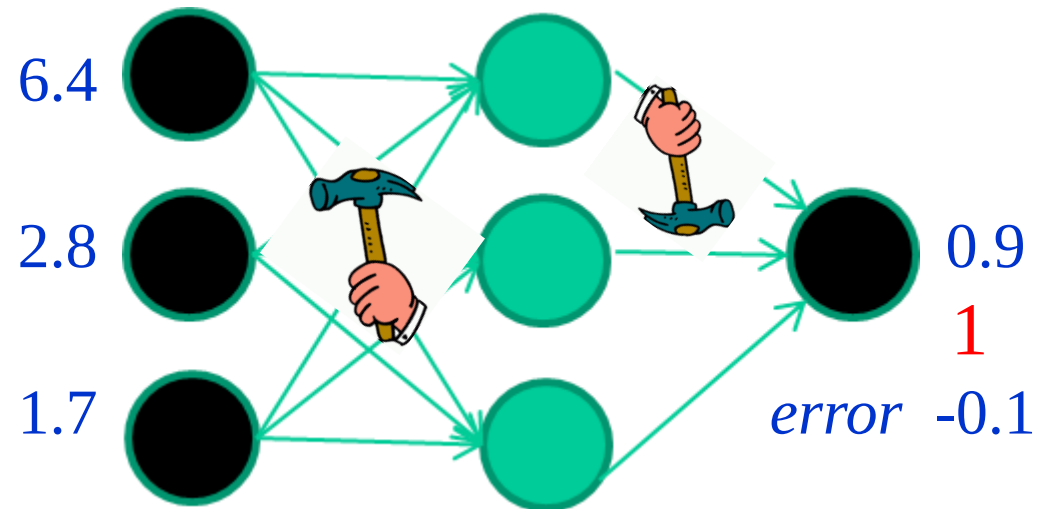
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Adjust weights based on error



Training data

Fields ***class***

1.4 2.7 1.9 0

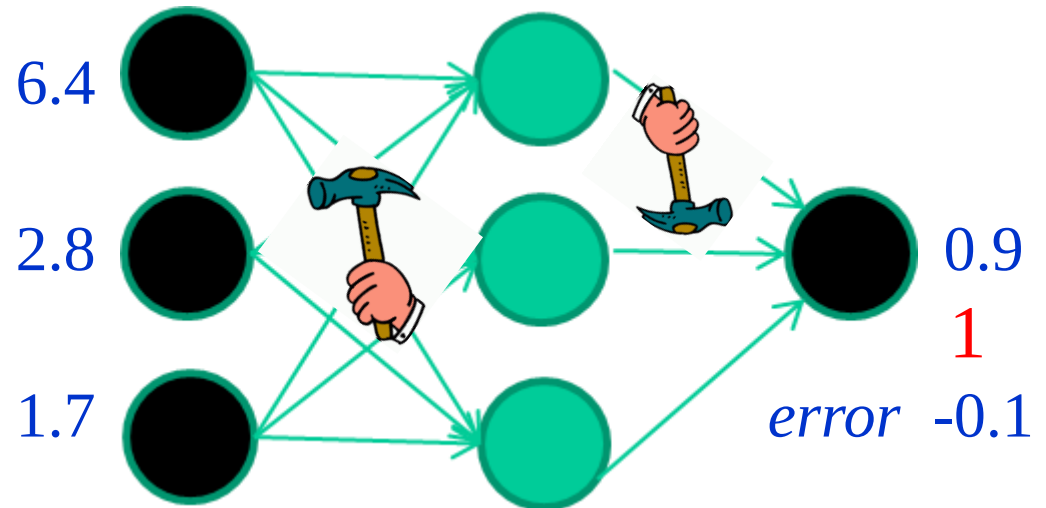
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

And so on



Repeat this thousands, maybe millions of times – each time taking a random training instance, and making slight weight adjustments

Algorithms for weight adjustment are designed to make changes that will reduce the error

'Back-prop' algorithm summary

◆ Initialise weights at random, choose a learning rate η

◆ Until network is trained:

◆ For each training example (input pattern and target outputs):

– Do forward pass through net (with fixed weights) to produce outputs

-assuming J hidden layer nodes and N inputs for a 2-layer MLP:

$$y_k = f\left(\sum_{j=0} w_{jk} o_j\right) \text{ where } o_j \text{ is output from each hidden node } j: o_j = f\left(\sum_{i=0}^N w_{ji} x_i\right)$$

– For each output unit k , compute deltas: $\delta_k = (y_{\text{target}_k} - y_k) y_k (1 - y_k)$

– For hidden units j (from last to first hidden layer, for the case of more than 1 hidden layer) compute deltas: $\delta_j = o_j (1 - o_j) \sum_k w_{jk} \delta_k$

– For all weights, change weight by gradient descent: $\Delta w_{ij} = \eta \delta_j y_i$

-Specifically, for the 2-layer MLP, for weight from input layer unit i to

hidden layer unit j , the weight changes by: $\Delta w_{ij} = \eta \delta_j x_i$

And, for weight from hidden layer unit j to output layer unit k , weight changes

$$\Delta w_{jk} = \eta \delta_k o_j$$

Improving performance

- Changing the number of layers and number of neurons in each layer.
- Variation in Transfer functions.
- Changing the learning rate.
- Training for longer times.
- Type of pre-processing and post-processing.